

Frozen Cumulative-Demand Prefill Comparison

Session D: exploratory evidence record

9 October 2026

Decision: retain fixed1024 and close this mapping in the tested operating domain. This record preserves a completed negative comparison. It establishes neither a new method contribution nor a general impossibility result. The overall research and submission objective remains incomplete; the historical manuscript is unchanged.

Question and intervention

Can a simple cumulative prefill-demand rule retain the generation benefit of small chunks without paying a larger waiting cost? Earlier fixed512 comparisons motivated this question: shorter generation spans did not translate into faster completion. The present experiment tests one frozen rule, not a tuned family of revisions.

The only controlled action is the aggregate number of prefill tokens permitted in one native scheduler step. Admission, request order, decoder progress, KV allocation, recovery, model precision, and operators remain common. For the native-order prefix of unfinished prompts that have already arrived, the rule computes

$$r(t) = \max_j \frac{\sum_{i \leq j} p_i^{\text{remaining}}}{a_j + 4 - t}.$$

It selects the smallest pure-prefill cap in $\{512, 1024, 2048\}$ whose predicted processing rate meets $r(t)$. The same frozen mixed-step cost model, multiplied by 1.2, supplies the denominator. Expired deadlines or an infeasible/invalid prediction select 2048 as a progress fallback; none of these fallbacks occurred in the two measured demand runs. This is an adaptation of a classical arrived-work demand principle¹, not a new scheduling principle or a deadline guarantee. Underfilling, changing decoder load, and future arrivals prevent transferring a fluid feasibility argument to this system.

Deployment and measurement

The runtime is in-process vLLM 0.26 with PyTorch 2.11 and CUDA 13, serving BF16 OLMoE-1B-7B-0924-Instruct on one RTX PRO 6000 Blackwell GPU. Normal memory allocation uses a 0.9 device-memory target and 588,032 usable KV tokens. The engine permits 192 sequences, a 4,096-token model length, and a 4,096-token native step budget. Prefix caching and asynchronous scheduling are disabled. No artificial small-KV limit, preemption, or recovery intervention is used.

All arms receive the same 256 open-loop requests: 128 QMSum long prompts and 128 GSM8K short prompts, with 538,597 prompt tokens and 114,688 declared output tokens. Arrivals end at approximately 31.87 seconds; observation continues through drain, with a 180-second timeout. EOS is ignored for this fixed-output diagnostic. Planned external arrivals define TTFT and completion latency, including submission lag. Token timestamps are host observations after synchronous `engine.step()` returns, not isolated GPU kernel times.

The frozen research contract requires $\text{TTFT} \leq 4$ seconds, each request's maximum successive-output gap ≤ 100 ms, and completion latency ≤ 20 seconds. Joint goodput U is the number meeting all three divided by the full elapsed service time, including waiting and drain. These thresholds have no established production justification. They were not changed after observing this comparison.

The order was fixed1024, demand, demand, fixed1024 after four warmups in a shared compilation domain. Warmups exercised all three fixed caps at their actual limits. No compilation or graph capture was recorded after measurement began. Each measured arm completed all 256 requests, with no failure, rejection, timeout, unfinished request, or preemption.

¹Yao, Demers, and Shenker, FOCS 1995, Sections 3–4: [author-hosted paper](#).

Complete-service results

Cell / policy	Qualified	U (req/s)	Elapsed (s)	TTFT p95 (s)	Flow p95 (s)	Gap p95 (ms)
00 / Fixed 1024	183/256	3.7100	49.326	2.033	23.042	57.928
01 / Demand	151/256	2.9722	50.804	3.348	24.932	56.311
02 / Demand	153/256	3.0138	50.767	3.362	24.889	62.508
03 / Fixed 1024	183/256	3.7078	49.355	2.057	23.116	66.928

Pair 1 compares demand cell 01 with fixed cell 00; pair 2 compares demand cell 02 with fixed cell 03. These are two reverse-order repetitions of one already-seen development trace, not independent workload confirmation. The figure and table are generated directly from the retained analysis JSON files.

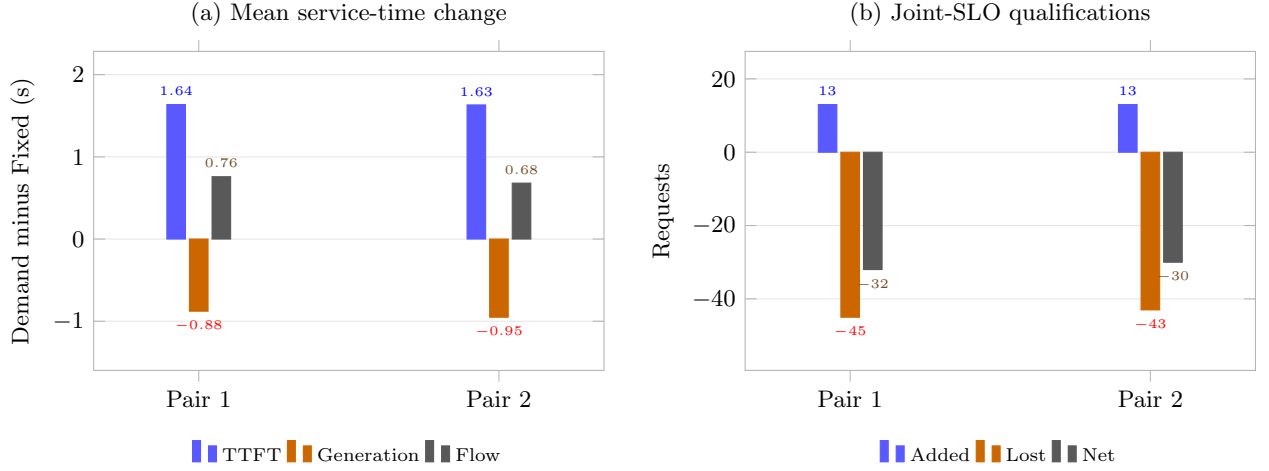


Figure 1: Demand minus contemporaneous fixed1024. Waiting increases exceed generation-span reductions. Qualified additions are outweighed by losses in both pairs. These are differences between full policy trajectories, not matched-state causal effects. All 256 requests enter each pair’s statistics.

Both demand runs improve qualification for the same 13 of the baseline’s 73 completion-only failures. They introduce 45 and 43 new completion-only failures, respectively. There are no new TTFT or maximum-gap violations: remaining inside the TTFT threshold does not make the additional waiting harmless to completion. For all 73 original failures, generation becomes shorter and TTFT becomes longer; their mean completion latency nevertheless increases in both pairs.

Goodput decreases by 19.89% and 18.72%. Raw output throughput decreases by 2.91% and 2.78%, while full elapsed time increases by 1.479 and 1.412 seconds. Thus the outcome is not a qualification gain masked by a longer denominator, nor a gain obtained by shortening drain. Both the qualified-request numerator and elapsed-time denominator move against demand. Per-request maximum-gap P95 improves by 2.79% and 6.60%; this benefit is retained in the report rather than treated as evidence of an overall win.

Execution evidence, interpretation, and limits

The rule actually changes execution on 727 and 726 steps: 724 binding reductions below 1024 in each run and 3/2 steps executing more than 1024 prefill tokens. There are no zero-prefill steps with a backlog. Total measured controller time is approximately 0.103 seconds per demand run versus 0.057 seconds for fixed1024; all of it remains included in elapsed time. Peak KV occupancy is approximately 62% for demand and 66% for fixed1024, so this comparison is not a capacity-eviction experiment.

The descriptive identity is completion latency = TTFT + first-to-last-output span + last-output-to-completion remainder. The last term is zero here. This identity does not assign causal shares to overlapping GPU work. Equal prompt/output counts also do not imply equal contents, routing, computation, or quality: 254 requests have different final token sequences in each paired comparison. Natural EOS and semantic task quality were not evaluated.

Decision and reproducibility

The result rejects this frozen mapping for this operating domain and joint objective. It does not refute the broader problem or establish fixed1024 as globally optimal. Niyama already combines deadlines and estimated remaining service; Sarathi-Serve and SLOWeave already address chunking and decode interference.² No full-system superiority over these neighbors is claimed. Positive-only follow-ups such as head-only or mean-work-matched controls are not launched to rescue this negative result. Further investment requires a separately justified question and new evidence.

Retained failure and commands. The preceding attempt, `demand53005_01`, stopped at the disk-space guard before model initialization: zero warmups and zero measured runs. Its records are retained as a resource failure, not a negative method result.

From `D_prefill_budget_20261004/continuation_20261007`:

```
python3 -B prefill_demand/analyze_demand.py demand53005_02 \
  --design prefill_demand_resume/design.json
python3 -B prefill_demand/decompose_service.py demand53005_02
python3 -B prefill_demand/render_results.py
cd demand53005_02/presentation
pdflatex -interaction=nonstopmode -halt-on-error report.tex
```

The sources are `demand53005_02/demand_results.json`, `demand53005_02/service_decomposition.json`, all eight retained raw files, and the run log. The frozen design is `prefill_demand_resume/design.json`; the original rule is `prefill_demand/controller.py`. Code hashes and full request migrations remain in the JSON and `RESULTS.md`. These commands reproduce analysis and this record; they do not launch a new GPU experiment.

²Primary sources: [Niyama](#), [Sarathi-Serve](#), [SLOWeave](#). See `NEIGHBORS.txt` for component boundaries.